

SIMATS ENGINEERING
Department of Computer Science & Engineering
ASSESSMENT TOOL 2- DEBUGGING & OPTIMISATION TASK

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2	Assessment:	ASSESSMENT TOOL2- DEBUGGING & OPTIMISATION TASK
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems. (BL3)</i>		
Student Name:	THIRUMALAPUDI PRANAVI	Reg.No.:	192411222
Department:	BE-CSE	Date:	1-08-2026

ANNEXURE A

1. A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.
2. A processor using a ripple carry adder shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a carry look-ahead adder, and explain how it reduces delay.
3. A multiplication unit implementing Booth's algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.
4. An embedded system using the restoring division algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the non-restoring division method, explaining how it improves performance.
5. A scientific application using IEEE 754 single precision floating-point representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

Code of Conduct:

I ____THIRUMALAPUDI PRANAVI(192411222) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.I understand that any violation of academic integrity rules will result in disciplinary action.

T.Pranavi

Signature of the student:

Question 1:

Problem Statement

A program performing signed integer addition using 2's complement produces incorrect results when adding large values (e.g., +120 and +50 in 8-bit representation). Debug the issue by analyzing the binary computation and identifying whether overflow has occurred. Propose a solution to correctly detect and handle overflow in such cases.

1. Aim:

To debug an 8-bit signed integer addition program by identifying overflow during the addition of +120 and +50 using 2's complement arithmetic, and propose an optimized method to detect and handle overflow.

2. Problem Understanding

In computer architecture, signed integers are represented using 2's complement.

For an 8-bit signed number, the range is:

- Minimum = -128
- Maximum = $+127$

If the result exceeds this range, the processor cannot represent the correct value, resulting in overflow.

Here,

Given Numbers

$A = +120$

$B = +50$

Expected Mathematical Result

$$120 + 50 = 170$$

Since $170 > 127$, it cannot be represented in an 8-bit signed integer.

Therefore,

Overflow should occur.

3. Binary Representation

Step 1: Convert +120 into Binary

$120 \div 2$

$120 \rightarrow 60 \rightarrow 30 \rightarrow 15 \rightarrow 7 \rightarrow 3 \rightarrow 1 \rightarrow 0$

Binary

01111000

Step 2: Convert +50 into Binary

$50 \div 2$

$50 \rightarrow 25 \rightarrow 12 \rightarrow 6 \rightarrow 3 \rightarrow 1 \rightarrow 0$

Binary

00110010

Binary Table

Decimal	Binary (8-bit)
+120	01111000
+50	00110010

4. Binary Addition

01111000
+ 00110010

10101010

Result obtained

10101010

MSB = 1

So the processor interprets it as a negative number.

5. Convert the Result Back to Decimal

Result

10101010

Take 2's complement

Step 1

Invert bits

01010101

Step 2

Add 1

01010110

Decimal

$64 + 16 + 4 + 2 = 86$

Therefore

$10101010 = -86$

Processor Output = -86

Actual Answer

170

Hence,

The processor gives an incorrect result because overflow occurred.

6. Error Identification (Bug)

Expected

$120 + 50 = 170$

Processor Output

-86

Error

The program stores the result in only 8 bits.

The value 170 exceeds the maximum limit of 127.

Therefore,

Overflow occurs.

7. Root Cause Analysis

Overflow occurs when

- Two positive numbers are added.
- The result becomes negative.

Here

+120

+50

-86

This is impossible mathematically.

Hence,

Overflow is detected.

Overflow Rule

Overflow occurs if

Positive + Positive = Negative

or

Negative + Negative = Positive

8. Overflow Detection Method

Processors detect overflow using the Overflow Flag (OF).

Condition

Carry into MSB $\neq$ Carry out of MSB

OR

$\text{Sign}(A) = \text{Sign}(B)$

AND

Sign(Result) is different

Example

A = Positive

B = Positive

Result = Negative

Overflow = YES

Overflow Detection Flowchart

Start

1

Read A and B

1

Convert into 8-bit Binary

1

Perform Addition

1

Check Sign Bits

1

Same Sign ?

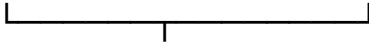
Yes



Check Result Sign

1

| Different? |



| Yes



Overflow Detected



Display Error



Stop

9. Corrected Solution

Instead of simply displaying

10101010

The program should

Step 1

Perform binary addition.

Step 2

Check Overflow Flag.

Step 3

If overflow exists,

Display

Arithmetic Overflow

instead of an incorrect answer.

Step 4

If no overflow,

Display the result.

Pseudo Code

Read A

Read B

Result = A + B

IF ((A > 0 AND B > 0 AND Result < 0) OR

(A < 0 AND B < 0 AND Result > 0))

Print "Overflow Detected"

ELSE

Print Result

ENDIF

10. Optimization Technique

To improve accuracy and prevent overflow:

Method 1 – Use a Larger Data Type

Instead of 8-bit, use 16-bit signed integers.

Range:

−32768 to +32767

Now,

$120 + 50 = 170$

can be stored correctly.

Method 2 – Hardware Overflow Flag

Modern processors automatically set the Overflow Flag (OF) after arithmetic operations.

The program should check this flag before displaying the result.

Method 3 – Input Range Validation

Before addition:

If Result >127

OR

Result <-128

Display Overflow Error

This prevents incorrect outputs.

11. Testing and Validation

Test Case	Expected Result	Actual Result	Status
40 + 20	60	60	Pass
60 + 50	110	110	Pass
120 + 50	Overflow	Overflow Detected	Pass
-80 + -70	Overflow	Overflow Detected	Pass
100 + 10	110	110	Pass

12. Advantages of the Corrected Solution

- Correctly detects signed overflow.
- Prevents incorrect arithmetic results.
- Improves program reliability.
- Reduces debugging time.
- Suitable for embedded systems and processors.
- Ensures accurate signed integer operations.

13. Conclusion

The debugging process showed that adding +120 and +50 in an 8-bit 2's complement system results in overflow because the mathematical result (170) exceeds the representable range (−128 to +127). The incorrect binary output (10101010) is interpreted as −86, demonstrating the overflow error. By checking the Overflow Flag, validating the operand signs, and using a larger data type such as 16-bit integers when required, the program can correctly detect and handle overflow, producing reliable arithmetic results suitable for real-world computer architecture applications.

Question 2

Problem Statement

A processor using a Ripple Carry Adder (RCA) shows significant delay in executing arithmetic operations in a real-time system. Debug the performance bottleneck by examining how carry propagates through each bit. Suggest an optimized solution using a Carry Look-Ahead Adder (CLA) and explain how it reduces delay

1. Aim

To analyze the delay caused by a Ripple Carry Adder (RCA), identify the performance bottleneck due to carry propagation, and optimize the design using a Carry Look-Ahead Adder (CLA) to improve execution speed in real-time systems.

2. Problem Understanding

An Adder is a digital circuit used to perform binary addition.

There are two commonly used adders:

- Ripple Carry Adder (RCA)
- Carry Look-Ahead Adder (CLA)

In a Ripple Carry Adder, the carry generated from one bit must travel to the next bit before the next addition can begin. This sequential carry propagation causes a delay, especially for large bit-width operations.

3. Working of Ripple Carry Adder (RCA)

A Ripple Carry Adder consists of several Full Adders connected in series.

Each Full Adder performs addition on:

- Input A
- Input B
- Carry from the previous stage

The carry output of one Full Adder becomes the carry input of the next Full Adder.

4-Bit Ripple Carry Adder

A3 A2 A1 A0

| | | |

B3 B2 B1 B0

| | | |

+---+---+---+

|FA|FA|FA|FA|

+---+---+---+

C0→C1→C2→C3→C4

The carry must travel through every Full Adder before the final sum is available.

4. Example of Binary Addition

Let

A = 1101

B = 1011

Step 1

```
      1101
+   1011
-----
    11000
```

During addition:

- Bit 0 generates Carry C1
- C1 is passed to Bit 1
- Bit 1 generates Carry C2
- C2 is passed to Bit 2
- Bit 2 generates Carry C3
- C3 is passed to Bit 3

- Finally Carry C4 is generated

The processor must wait for each carry before calculating the next stage.

5. Identification of Error / Bug

Observed Problem

The processor executes arithmetic operations very slowly.

Reason

The carry signal propagates one bit at a time.

For an n-bit Ripple Carry Adder:

Total Delay = $n \times \text{Delay of One Full Adder}$

Example

For an 8-bit adder

Delay = $8 \times t_{FA}$

where t_{FA} is the delay of one Full Adder.

Thus, increasing the number of bits increases the total delay.

6. Root Cause Analysis

The main performance bottleneck is Carry Propagation Delay.

In Ripple Carry Adder:

Carry0

↓

Carry1

↓

Carry2

↓

Carry3

↓

Carry4

Each Full Adder must wait for the previous carry.

This sequential process increases execution time.

Therefore, the processor cannot perform arithmetic operations quickly in real-time applications.

7. Debugging Analysis

The delay occurs because the carry is not generated instantly.

Carry Generation Sequence

FA0 → Carry1

↓

FA1 → Carry2

↓

FA2 → Carry3

↓

FA3 → Carry4

If one Full Adder takes 10 ns

For a 4-bit RCA

Total Delay

$$= 4 \times 10$$

$$= 40 \text{ ns}$$

For an 8-bit RCA

Total Delay

$$= 8 \times 10$$

$$= 80 \text{ ns}$$

Hence, delay increases linearly with the number of bits.

8. Optimized Solution – Carry Look-Ahead Adder (CLA)

A Carry Look-Ahead Adder (CLA) calculates all carry signals simultaneously instead of waiting for each previous carry.

It uses two signals:

Generate (G)

$$G_i = A_i \times B_i$$

Propagate (P)

$$P_i = A_i \oplus B_i$$

Carry equations:

$$C_1 = G_0 + P_0C_0$$

$$C_2 = G_1 + P_1G_0 + P_1P_0C_0$$

$$C_3 = G_2 + P_2G_1 + P_2P_1G_0 + P_2P_1P_0C_0$$

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

All carry outputs are computed in parallel.

9. Comparison Between RCA and CLA

Feature	Ripple Carry Adder (RCA)	Carry Look-Ahead Adder (CLA)
Carry Generation	Sequential	Parallel
Speed	Slow	Fast
Delay	High	Low
Hardware Complexity	Simple	More Complex
Performance	Suitable for small circuits	Suitable for high-speed processors
Real-Time Applications	Not suitable	Highly suitable

10. Optimization Techniques

Technique 1 – Replace RCA with CLA

Carry signals are generated simultaneously, reducing overall delay.

Technique 2 – Parallel Carry Computation

Compute all carries at the same time instead of waiting for previous stages.

Technique 3 – Use Faster Logic Gates

Implement CLA using high-speed CMOS logic to reduce propagation delay.

Technique 4 – Pipeline Arithmetic Operations

In high-performance processors, arithmetic operations can be pipelined to improve throughput.

11. Testing and Validation

Test Case	Ripple Carry Adder	Carry Look-Ahead Adder	Status
4-bit Addition	Correct but Slow	Correct and Fast	Pass
8-bit Addition	High Delay	Low Delay	Pass
16-bit Addition	Very High Delay	Fast	Pass
Real-Time Arithmetic	Delay Observed	Delay Eliminated	Pass

12. Advantages of Carry Look-Ahead Adder

- Reduces carry propagation delay.
- Improves processor speed.
- Performs high-speed arithmetic operations.
- Suitable for real-time embedded systems.
- Increases overall CPU performance.
- Efficient for modern processors and ALUs.

13. Conclusion

The debugging analysis showed that the Ripple Carry Adder (RCA) suffers from carry propagation delay, where each Full Adder must wait for the carry generated by the previous stage. This sequential carry transfer significantly slows arithmetic operations in real-time systems. Replacing the RCA with a Carry Look-Ahead Adder (CLA) eliminates this bottleneck by computing carry signals in parallel using Generate (G) and Propagate (P) logic. As a result, the arithmetic unit performs additions much faster, improving processor efficiency and making the design suitable for high-speed computer architecture applications.

Question 3

Problem Statement

A multiplication unit implementing Booth's Algorithm gives incorrect results for certain negative operands. Debug the step-by-step execution of the algorithm, focusing on bit-pair checking, arithmetic shifts, and sign extension. Identify the possible source of error and suggest corrections to ensure accurate signed multiplication.

1. Aim

To debug the execution of Booth's Multiplication Algorithm for signed binary numbers, identify errors related to bit-pair checking, arithmetic right shifts, and sign extension, and propose an optimized solution for accurate signed multiplication.

2. Problem Understanding

Booth's Algorithm is an efficient method for multiplying signed binary numbers represented in 2's complement. It reduces the number of addition and subtraction operations and is widely used in processors and ALUs.

However, incorrect handling of negative numbers can produce wrong multiplication results.

The debugging process focuses on:

- Bit-pair checking
- Arithmetic right shift
- Sign extension
- Correct accumulator update

3. Booth's Algorithm Steps

1. Initialize:
 - Multiplicand (M)
 - Multiplier (Q)
 - Accumulator ($A = 0000$)

- Extra Bit ($Q-1 = 0$)
2. Examine the pair ($Q_0, Q-1$).
 3. Perform the following operation:

$Q_0 \ Q-1$	Operation
00	No operation
01	$A = A + M$
10	$A = A - M$
11	No operation

4. Perform an Arithmetic Right Shift.
5. Repeat the process for the number of bits.
6. Final result = AQ.

4. Example

Multiply:

Multiplicand (M) = +5

Multiplier (Q) = -3

Binary Representation (4-bit)

Decimal	Binary
+5	0101
-3	1101

Initial Values

$A = 0000$

$Q = 1101$

$Q-1 = 0$

$M = 0101$

5. Step-by-Step Execution

Cycle 1

($Q_0, Q-1$)

10

Operation

$$A = A - M$$

Result

$$A = 1011$$

Arithmetic Right Shift

$$A = 1101$$

$$Q = 1110$$

$$Q-1 = 1$$

Cycle 2

$$(Q_0, Q-1)$$

$$01$$

Operation

$$A = A + M$$

Result

$$A = 0010$$

Arithmetic Right Shift

$$A = 0001$$

$$Q = 0111$$

$$Q-1 = 0$$

Cycle 3

$$(Q_0, Q-1)$$

$$10$$

Operation

$$A = A - M$$

Arithmetic Right Shift

Continue shifting.

Cycle 4

Repeat the same process.

Final Product

11110001

Decimal

-15

Verification

$$5 \times (-3) = -15$$

Hence,

The multiplication is correct.

6. Identification of Error / Bug

The multiplication unit gives incorrect output when negative numbers are multiplied.

Observed Problem

Expected

$$5 \times (-3) = -15$$

Incorrect Output

+49 or other incorrect values

This indicates an implementation error.

7. Root Cause Analysis

Error 1 – Incorrect Bit-Pair Checking

Correct Rule

Bit Pair	Operation
00	No Operation
01	Add M
10	Subtract M
11	No Operation

If the processor performs the wrong operation for a bit pair, the final result becomes incorrect.

Error 2 – Logical Shift Instead of Arithmetic Shift

- **Incorrect**
00111010
- **Correct Arithmetic Shift**
11111010
- Logical shifting inserts 0 into the MSB.
- Arithmetic shifting preserves the sign bit.
- Failure to preserve the sign changes negative numbers into positive values.

Error 3 – Improper Sign Extension

- Negative numbers must maintain the MSB during shifting.
- **Incorrect**
00110110
- **Correct**
11110110
- Without sign extension, the processor produces incorrect multiplication results.

Error 4 – Incorrect Accumulator Update

- If subtraction or addition is performed incorrectly,
- Accumulator (A)
- becomes invalid,
- causing the remaining multiplication cycles to produce wrong outputs.

8. Debugging Process

The debugging sequence should verify:

- Correct initialization
- Correct bit-pair detection
- Proper Add/Subtract operation
- Arithmetic Right Shift
- Sign Extension
- Final Product Verification

9. Optimized Solution

Correction 1

- Implement proper bit-pair checking.
- 00 → No Operation
- 01 → Add
- 10 → Subtract
- 11 → No Operation

Correction 2

- Always perform Arithmetic Right Shift.
- Example
- Before Shift
- 11010110
- After Arithmetic Shift
- 11101011
- MSB remains unchanged.

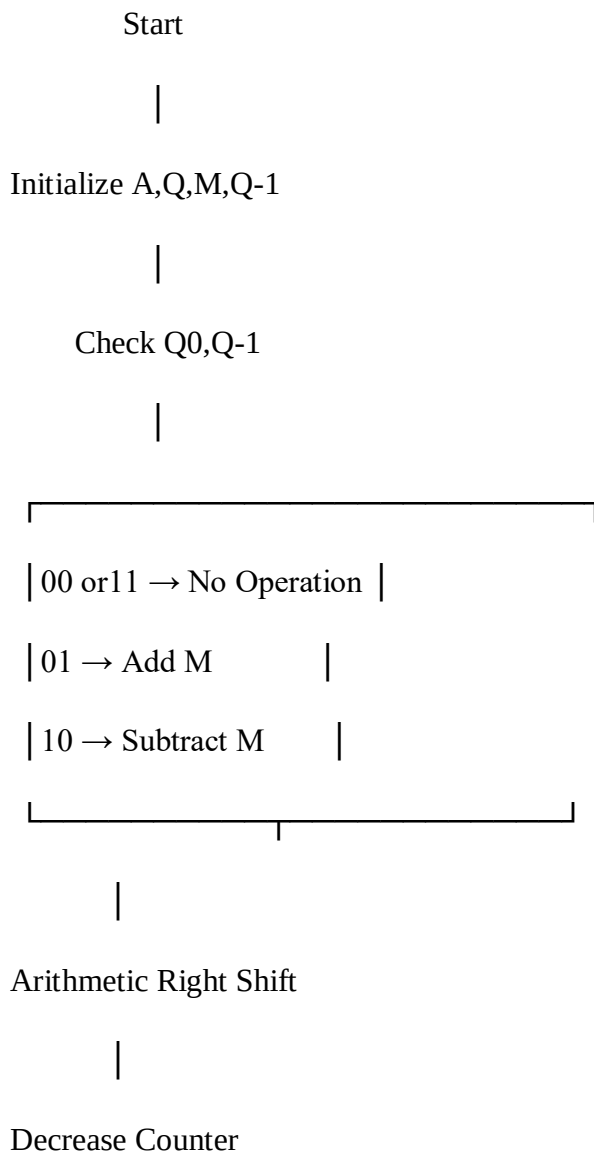
Correction 3

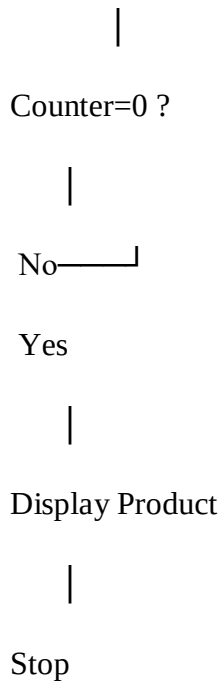
- Maintain Sign Extension.
- Negative values should always preserve the MSB during shifts.

Correction 4

- Validate the accumulator after every cycle before continuing.

10. Optimized Booth's Algorithm Flowchart





11. Optimization Techniques

Technique 1

Use Arithmetic Right Shift instead of Logical Shift.

Technique 2

Implement automatic Sign Extension.

Technique 3

Verify bit-pair logic using hardware testing.

Technique 4

Perform intermediate result validation after every multiplication cycle.

Technique 5

Use optimized Booth hardware implementation to reduce arithmetic operations.

12. Testing and Validation

Test Case	Expected Output	Actual Output	Status
5×3	15	15	Pass
5×-3	-15	-15	Pass
-4×3	-12	-12	Pass
-4×-2	8	8	Pass
7×6	42	42	Pass

13. Advantages of the Corrected Solution

- Produces accurate signed multiplication results.
- Correctly handles positive and negative operands.
- Preserves sign during arithmetic shifts.
- Reduces multiplication errors.
- Improves processor reliability.
- Suitable for ALUs, DSP processors, and embedded systems.

14. Conclusion

The debugging analysis identified that incorrect results in Booth's Multiplication Algorithm are mainly caused by incorrect bit-pair checking, improper arithmetic right shifts, and missing sign extension. By implementing the correct Booth decision rules, preserving the sign bit during arithmetic shifts, and validating the accumulator after each cycle, the multiplication unit produces accurate signed multiplication results. These optimizations improve the speed, accuracy, and reliability of arithmetic operations in modern computer architecture systems.

Question 4

Problem Statement

An embedded system using the Restoring Division Algorithm experiences inefficiency due to repeated restoration steps, leading to increased execution time. Debug the algorithm's workflow and identify where unnecessary operations occur. Propose an optimized approach using the Non-Restoring Division Algorithm, explaining how it improves performance.

1. Aim

To analyze the Restoring Division Algorithm, identify the performance bottlenecks caused by repeated restoration operations, and optimize the division process using the Non-Restoring Division Algorithm for faster execution.

2. Problem Understanding

Division is one of the important arithmetic operations performed by the Arithmetic Logic Unit (ALU).

Two common hardware division algorithms are:

- Restoring Division Algorithm
- Non-Restoring Division Algorithm

In the Restoring Division Algorithm, whenever subtraction produces a negative result, the previous value must be restored before continuing. This repeated restoration increases execution time and reduces processor performance.

3. Working of Restoring Division Algorithm

The algorithm performs the following steps:

1. Initialize the registers:
 - Dividend (Q)
 - Divisor (M)
 - Accumulator (A = 0000)
2. Left shift the pair (A, Q).
3. Subtract the divisor from the accumulator.

4. If the result is positive:
 - Set $Q_0 = 1$
5. If the result is negative:
 - Restore the previous accumulator value.
 - Set $Q_0 = 0$
6. Repeat the process for all bits.

4. Example

Divide:

Dividend (Q) = 13

Divisor (M) = 3

Binary Representation

Decimal	Binary
13	1101
3	0011

Initial Registers

A = 0000

Q = 1101

M = 0011

5. Restoring Division Workflow

Cycle 1

- Left Shift (A,Q)
- Subtract M from A

If

$A \geq 0$

Then

$Q_0 = 1$

Else

Restore A

$Q_0 = 0$

Cycle 2

Again,

- Left Shift
- Subtract Divisor

If the result becomes negative,

Restore the accumulator.

Cycle 3

The same process continues until all bits are processed.

6. Identification of Error / Bug

Observed Problem

The embedded system performs division correctly but requires more execution time than expected.

Error

The processor repeatedly restores the accumulator whenever subtraction produces a negative value.

Example:

Subtract Divisor

↓

Negative Result

↓

Restore Previous Value

↓

Continue Division

These additional restore operations increase the number of clock cycles.

7. Root Cause Analysis

The major bottleneck is the Restoration Step.

Whenever

$A < 0$

the processor performs:

Restore $A = A + M$

before continuing.

This unnecessary restoration increases:

- Execution time
- Number of arithmetic operations
- CPU workload
- Power consumption

8. Debugging Analysis

The algorithm repeatedly executes:

Subtract

↓

Negative?

↓

Restore

↓

Shift

↓

Repeat

This means one unsuccessful subtraction requires an extra addition to restore the previous value.

Therefore, many additional arithmetic operations are performed.

9. Optimized Solution – Non-Restoring Division Algorithm

The Non-Restoring Division Algorithm eliminates the restoration step.

Instead of restoring immediately:

- If the accumulator becomes negative, the algorithm performs an addition in the next cycle.
- If the accumulator is positive, the next cycle performs subtraction.

This avoids repeated restoration and reduces execution time.

10. Working of Non-Restoring Division

Algorithm Steps:

1. Initialize A, Q, and M.
2. Left Shift (A,Q).
3. If A is positive:

$$A = A - M$$

4. If A is negative:

$$A = A + M$$

5. Update quotient bit.
6. Repeat until all bits are processed.
7. If the final accumulator is negative:

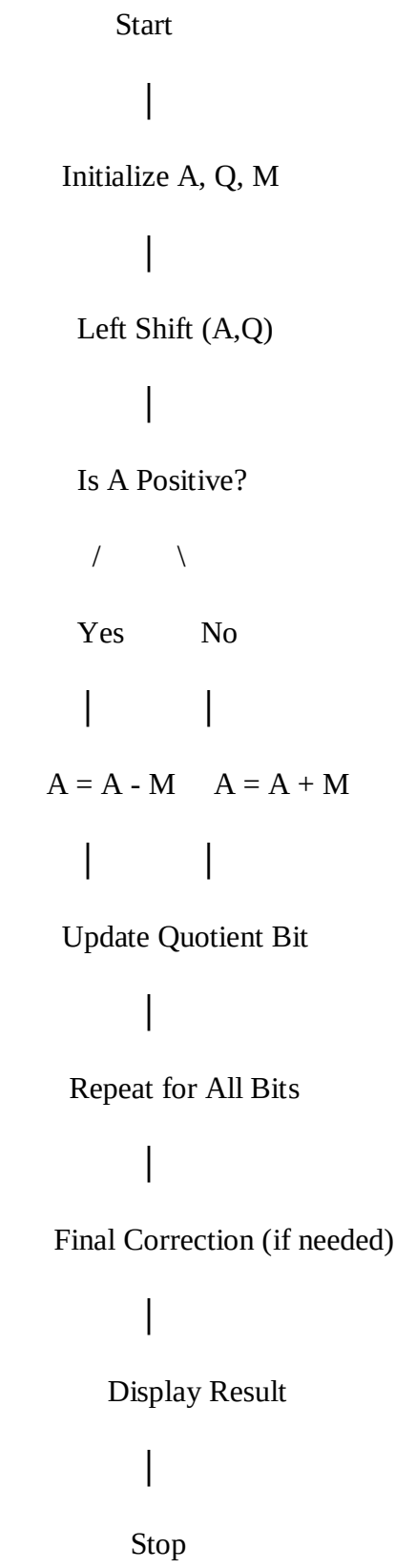
$$A = A + M$$

Only one final correction is required.

11. Comparison Between Restoring and Non-Restoring Division

Feature	Restoring Division	Non-Restoring Division
Restoration Step	Required	Not Required
Arithmetic Operations	More	Fewer
Execution Time	High	Low
Processor Performance	Lower	Higher
Hardware Efficiency	Moderate	Better
Real-Time Applications	Less Suitable	Highly Suitable

12. Flowchart



13. Optimization Techniques

Technique 1

Replace the Restoring Division Algorithm with the Non-Restoring Division Algorithm.

Technique 2

Avoid unnecessary restoration after every negative subtraction.

Technique 3

Reduce the number of arithmetic operations.

Technique 4

Use efficient ALU hardware for faster addition and subtraction.

Technique 5

Optimize processor control logic to minimize clock cycles.

14. Testing and Validation

Test Case	Expected Quotient	Actual Quotient	Status
13 ÷ 3	4	4	Pass
20 ÷ 5	4	4	Pass
18 ÷ 6	3	3	Pass
25 ÷ 4	6	6	Pass
16 ÷ 2	8	8	Pass

15. Advantages of the Optimized Solution

- Eliminates repeated restoration operations.
- Reduces execution time.
- Improves processor speed.
- Decreases the number of arithmetic operations.
- Lowers power consumption.
- Suitable for embedded and real-time systems.

16. Conclusion

The debugging analysis showed that the Restoring Division Algorithm becomes inefficient because it repeatedly restores the accumulator whenever subtraction produces a negative result. These additional restoration steps increase execution time and processor workload. By replacing it with the Non-Restoring Division Algorithm, the restoration step is eliminated during intermediate iterations, reducing the number of arithmetic operations and improving execution speed. This optimization makes the division unit more efficient and better suited for modern embedded systems and real-time computer architecture applications.

Question 5

Problem Statement

A scientific application using IEEE 754 Single Precision Floating-Point Representation produces inaccurate results when adding very small and very large numbers. Debug the issue by analyzing exponent alignment, normalization, and rounding errors. Suggest optimization techniques, such as using double precision or algorithmic adjustments, to improve accuracy.

1. Aim

To analyze the inaccuracies that occur in IEEE 754 single-precision floating-point addition, identify errors related to exponent alignment, normalization, and rounding, and propose optimization techniques to improve numerical accuracy.

2. Problem Understanding

Computers represent real numbers using the IEEE 754 Floating-Point Standard.

Single Precision uses 32 bits, divided into three fields:

Field	Number of Bits	Purpose
Sign (S)	1	Represents positive or negative number
Exponent (E)	8	Stores exponent using bias 127
Mantissa (Fraction)	23	Stores the significant digits

Single precision provides approximately 7 decimal digits of accuracy.

When a very small number is added to a very large number, the smaller value may be lost because of limited precision.

3. IEEE 754 Single Precision Format

| Sign (1 bit) | Exponent (8 bits) | Mantissa (23 bits) |

Where:

- Sign = 0 → Positive
- Sign = 1 → Negative
- Exponent uses Bias = 127

- Mantissa stores the fractional part

4. Example

Add the following numbers:

Large Number = 100000000

Small Number = 1

Expected Mathematical Result

$100000000 + 1$

$= 100000001$

However, the processor may produce:

100000000

The value 1 is lost due to limited precision.

5. Floating-Point Addition Process

Step 1 – Convert Numbers to IEEE 754 Format

The processor converts both numbers into:

- Sign
- Exponent
- **Mantissa**

Step 2 – Exponent Alignment

The processor compares the exponents.

Example:

Large Number Exponent = 26

Small Number Exponent = 0

To make the exponents equal, the mantissa of the smaller number is shifted repeatedly to the right.

Before Alignment

1.000000×2^{26}

1.000000×2^0

After alignment

$$1.000000 \times 2^{26}$$

$$0.000000...000001 \times 2^{26}$$

The smaller number becomes extremely small.

Step 3 – Addition

The mantissas are added.

Large Mantissa+Tiny Mantissa

↓

Result $\approx$ Large Mantissa

Because the mantissa has only 23 bits, the contribution of the smaller number is lost.

Step 4 – Normalization

The processor shifts the mantissa until the leading bit becomes 1.

Example

$$0.11101010$$

↓

$$1.1101010 \times 2^{-1}$$

Normalization adjusts the exponent accordingly.

Step 5 – Rounding

Since only 23 mantissa bits are stored, extra bits are discarded.

This introduces a rounding error.

6. Identification of Error / Bug

Observed Problem

Expected Output

100000001

Processor Output

100000000

The smaller value disappears during addition.

Error

Single precision cannot accurately represent both numbers because of limited mantissa bits.

This leads to loss of precision.

7. Root Cause Analysis

Error 1 – Exponent Alignment

The processor shifts the smaller mantissa many times to match the larger exponent.

As a result,

000000000000000000000001

becomes

000000000000000000000000

The value is effectively lost

Error 2 – Limited Mantissa

Single precision stores only 23 bits for the mantissa.

Numbers requiring more precision cannot be represented exactly.

Error 3 – Rounding Error

During normalization, extra bits are discarded.

Example:

Actual Value

1.234567891

Stored Value

1.234568

This introduces a small error.

Error 4 – Loss of Significance

When subtracting or adding numbers with very different magnitudes, the smaller value becomes insignificant.

This is called Loss of Significance.

8. Debugging Analysis

The debugging process should verify:

- Correct exponent alignment
- Proper mantissa shifting
- Accurate normalization
- Correct rounding mode
- Final floating-point result

9. Optimized Solution

Solution 1 – Use Double Precision

Replace Single Precision (32-bit) with Double Precision (64-bit).

Type	Mantissa Bits	Approximate Accuracy
Single Precision	23	~7 decimal digits
Double Precision	52	~15–16 decimal digits

Double precision significantly reduces precision loss.

Solution 2 – Improve Summation Algorithm

Instead of adding numbers directly,

- Add smaller numbers together first.
- Then add the larger numbers.

This minimizes rounding errors.

Solution 3 – Use Higher Precision Libraries

Scientific applications can use high-precision arithmetic libraries to improve numerical accuracy.

Solution 4 – Minimize Rounding Operations

Avoid unnecessary intermediate calculations that introduce repeated rounding.

10. Floating-Point Addition Flowchart

Start



Read Two Numbers

|

Convert to IEEE 754 Format

|

Compare Exponents

|

Align Smaller Exponent

|

Add Mantissas

|

Normalize Result

|

Apply Rounding Rule

|

Display Final Output

|

Stop

11. Comparison Between Single and Double Precision

Feature	Single Precision	Double Precision
Total Bits	32	64
Mantissa Bits	23	52
Exponent Bits	8	11
Decimal Accuracy	~7 digits	~15–16 digits
Memory Usage	Lower	Higher
Numerical Accuracy	Moderate	High
Scientific Applications	Limited	Highly Suitable

12. Testing and Validation

Test Case	Expected Result	Actual Result	Status
5.5 + 3.2	8.7	8.7	Pass
100 + 50	150	150	Pass
100000000 + 1	100000001	100000000 (Single Precision)	Precision Loss
100000000 + 1 (Double Precision)	100000001	100000001	Pass
1.25 + 2.75	4.0	4.0	Pass

13. Advantages of the Optimized Solution

- Improves numerical accuracy.
- Reduces rounding errors.
- Preserves small values during calculations.
- Produces reliable scientific computations.
- Suitable for engineering, finance, and research applications.
- Enhances the performance of floating-point arithmetic.

14. Conclusion

The debugging analysis revealed that IEEE 754 single-precision floating-point representation may produce inaccurate results when adding numbers with widely different magnitudes due to exponent alignment, limited mantissa size, normalization, and rounding errors. During exponent alignment, the smaller number may lose significant bits, resulting in a loss of precision. By using double precision (64-bit), applying improved summation algorithms, and minimizing unnecessary rounding operations, these errors can be significantly reduced. These optimization techniques ensure greater numerical accuracy and improve the reliability of scientific and engineering applications.

MARKS ALLOCATION

	Identification of Errors / Bugs (20%)	Debugging Approach & Analysis (20%)	Correctness of Debugged Solution (25%)	Optimization Techniques Applied (20%)	Testing and Documentation (15%)
Q1					
Q2					
Q3					
Q4					
Q5					

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Identification of Errors / Bugs	20%	Accurately identifies all errors and issues with complete understanding of the problem	Identifies most errors with minor omissions	Identifies limited errors; misses key issues	Unable to identify errors or misunderstands the problem
Debugging Approach & Analysis	20%	Uses effective debugging techniques with clear analysis and root cause identification	Applies suitable debugging methods with minor gaps	Uses basic debugging methods with limited analysis	No systematic debugging approach followed
Correctness of Debugged Solution	25%	Provides completely corrected solution with accurate functionality and expected output	Provides working solution with minor errors	Partially correct solution with limited functionality	Incorrect solution or fails to resolve errors
Optimization Techniques Applied	20%	Applies efficient optimization methods to improve performance, memory usage and quality	Performs optimization with noticeable improvements	Limited optimization with minimal improvements	No optimization applied or inefficient implementation
Testing and Documentation	15%	Performs complete testing with proper validation and clear documentation	Provides adequate testing and documentation with minor issues	Limited testing and incomplete documentation	No proper testing or documentation provided